PICKGUARD AI

Evidence-Grounded AI Agent for Fulfilment Centre Pick Exception Resolution

Project: PickGuard AI Capstone Full-Stack System	**Team: Vision Forge**
Domain: Logistics / Fulfilment Operations	**Status:** All 12 Phases Completed & Verified (93 Pytest Passing)

Executive Summary

PickGuard AI is a production-grade, evidence-grounded agentic AI assistant engineered to resolve picking exceptions in high-volume e-commerce fulfilment centres. Traditional chatbots hallucinate warehouse state, invent operational policies, or inadvertently issue dangerous automated inventory mutations. PickGuard AI solves this with a strict multi-layered architecture where **raw facts are retrieved exclusively via deterministic Python tools**, standard procedures are grounded through **ChromaDB RAG**, and decisions are governed by a **deterministic safety gate** featuring real **LangGraph interrupt() checkpoints**.

Master Capstone Roadmap — All 12 Phases

Phase	System Milestone	Technical Implementation	Capstone Rubric Value
Phase 1	Project Setup & Architecture	Directory structure, virtualenv, FastAPI health, TypedDict StateGraph schema.	**Foundation**
Phase 2	Synthetic Warehouse Data	locations.csv, inventory.csv, pick_tasks.csv, incidents.csv (24 bins, 20 items, 20 tasks).	**Realistic Problem**
Phase 3	Deterministic Tools Layer	get_inventory, get_pick_task, get_location, search_similar_incidents, escalate_to_lead.	**Tool-Use Layer**
Phase 4	SOP Knowledge Base + RAG	6 SOP markdown docs, recursive chunking, ChromaDB vectorstore, metadata provenance.	**Grounding**
Phase 5	LangGraph State & Workflow	13-node StateGraph, deterministic conditional routing, MemorySaver checkpointer.	**Agent Architecture**
Phase 6	Controlled LLM Reasoning	Pydantic structured output, Groq / Ollama / Mimic multi-tier provider fallback routing.	**Generative AI**
Phase 7	Evidence Fusion & Next-Best Action	Cross-checks physical vs system inventory, conflict detection, action boundary enforcement.	**Unique Innovation**
Phase 8	Safety Gate & Human interrupt()	Real LangGraph interrupt() checkpoint, Command(resume=...) state resumption, loop limits.	**HITL & Responsible AI**
Phase 9	FastAPI Backend REST Services	/api/v1/agent/run, /api/v1/agent/{run_id}/review, CORS, Pydantic schemas, audit logging.	**Working System**
Phase 10	Operator Dashboard & Modern UI	React 18 + Vite + TypeScript, glassmorphism UI, scenario test cards, interrupt modal.	**Interactive Demo**
Phase 11	Automated Evaluation Suite	93 Pytest automated test cases covering RAG, tools, agent edges, safety, API, and E2E.	**Rubric Verification**
Phase 12	Final Documentation & Viva Prep	Architecture diagrams, model cards, 1-min & 3-min pitch scripts, 35+ viva Q&As.;	**Capstone Submission**

Detailed Phase-by-Phase Technical Breakdown

Phase 1: Project Setup & Core Architectural Foundation

• Initialized Python 3.12 environment with strict dependency pinning (LangGraph 0.2.x, LangChain 0.3.x, FastAPI, ChromaDB, HuggingFace).
• Established clean layered architecture separating API, Graph, Tools, RAG, Models, Policy, and Data layers.
• Implemented initial TypedDict state schema with health check endpoints verifying zero runtime startup regressions.

Phase 2: Synthetic Fulfilment Centre Data & Exception Taxonomy

• Built comprehensive synthetic datasets reflecting realistic fulfilment operations: 24 storage bins (`locations.csv`), 20 product SKUs (`inventory.csv`), 20 operational pick tasks (`pick_tasks.csv`), and 20 historical exception logs (`incidents.csv`).
• Established standard exception taxonomy: `MISSING_ITEM`, `QUANTITY_MISMATCH`, `BARCODE_FAILURE`, `WRONG_ITEM_IN_BIN`, `DAMAGED_ITEM`, and `LOCATION_DISCREPANCY`.

Phase 3: Deterministic Warehouse Tools Layer

• Implemented 5 pure Python deterministic tools with Pydantic contract validation:
- get_inventory(item_id, location_id): Validates physical vs system units and lot status.
- get_pick_task(task_id): Retrieves expected quantity, bin, and status.
- get_location(location_id): Returns zone and adjacent neighboring bins for scan sweeps.
- search_similar_incidents(exception_type, limit): Retrieves past resolution precedence.
- escalate_to_lead(task_id, reason): Creates formal supervisor escalation records.

Phase 4: Standard Operating Procedures (SOP) Knowledge Base & RAG

• Authored 6 standard operating procedure documents in Markdown covering step-by-step warehouse resolution rules.
• Configured LangChain `RecursiveCharacterTextSplitter` (chunk size 400, overlap 50) and `all-MiniLM-L6-v2` embeddings.
• Persistent ChromaDB vector database with source provenance metadata tracking chunk ID, file path, and title.

Phase 5: LangGraph StateGraph & Multi-Node Workflow

• Constructed compiled `StateGraph` consisting of 13 interconnected nodes:
START → parse_operator_input → classify_exception → retrieve_operational_data → retrieve_sop_evidence → retrieve_historical_evidence → build_evidence_package → llm_reasoning_node → evidence_fusion_node → evaluate_safety_policy → human_review_gate → apply_final_decision → END.
• Attached `MemorySaver` checkpointer supporting state persistence and pause/resume execution across threads.

Phase 6: Controlled LLM Reasoning & Multi-Tier Provider Routing

• Integrated Pydantic-grounded JSON reasoning schema enforcing separation of observed facts from inferences.
• Built resilient multi-tier provider abstraction:
1. **Groq Provider** (Ultra-fast cloud inference with Llama-3.3-70B)
2. **Ollama Provider** (Local offline inference with Llama3)
3. **Mimic Provider** (Deterministic offline fallback guaranteeing 100% demo reliability without API keys).

Phase 7: Evidence Fusion, Conflict Detection & Action Boundary

• **Evidence Fusion Engine:** Cross-checks physical operator observations against database records, flagging quantity discrepancies and missing barcode records.
• **Action Boundary Policy:** Strictly differentiates between *Recommendation* and *Execution*. Hazardous mutating actions (`ADJUST_QUANTITY`, `UPDATE_INVENTORY`, `MARK_DAMAGED`) are automatically tagged `BLOCKED`.

Phase 8: Safety Gate & Real LangGraph interrupt() Checkpoints

• Implemented real from langgraph.types import interrupt, Command.
• High-risk actions automatically trigger interrupt(payload), pausing workflow execution.
• Resumed via Command(resume={'decision': 'APPROVE' | 'REJECT' | 'REQUEST_MORE_EVIDENCE' | 'ESCALATE'}).
• Loop safety guard enforces max 2 review attempts before mandatory escalation.

Phase 9: FastAPI Enterprise Backend REST Services

• Built production REST endpoints: POST /api/v1/agent/run, GET /api/v1/agent/{run_id}, POST /api/v1/agent/{run_id}/review, GET /api/v1/system/status.
• Integrated CORS middleware and Pydantic request/response validation schemas.

Phase 10: Modern React Operator Dashboard & UI Redesign

• Modernized React 18 + Vite + TypeScript frontend with dark glassmorphism design system (`#050814` void palette, gradient mesh, glowing borders).
• Includes quick-load scenario cards, Claude-style chat prompt, interactive tabbed evidence explorer, and full-screen human review modal.

Phase 11: Comprehensive Evaluation & Pytest Test Suite

• Engineered **93 automated test cases** across 28 test suites in `backend/tests/`.
• Tests cover: RAG retrieval precision, tool contracts, graph routing edges, prompt injection safety, multi-signal exceptions, human review approval/rejection/evidence loops, and full end-to-end API workflows. **All 93 tests pass cleanly.**

Phase 12: Final Capstone Documentation & Viva Readiness

• Completed comprehensive documentation: Architecture blueprints (`docs/architecture.md`), responsible AI framework (`docs/responsible_ai.md`), performance benchmark reports, 1-minute elevator pitch, 3-minute demo script, and 35+ viva examination questions.

Capstone Evaluation Rubric Mapping

How PickGuard AI satisfies each required evaluation criterion for top-tier capstone distinction:

Evaluation Criterion	PickGuard AI Implementation Evidence	Status
Problem Complexity	Real-world fulfilment centre pick exception resolution involving multi-bin search, barcode scanner faults, and quantity discrepancies.	**100% Satisfied**
Agent Architecture	Compiled 13-node LangGraph StateGraph with conditional edge routing, checkpointer state recovery, and loop safety limits.	**100% Satisfied**
Tool-Use & Grounding	Zero raw LLM factual lookup. 5 deterministic warehouse tools + ChromaDB vector RAG retrieval with source provenance.	**100% Satisfied**
LLM Reasoning & Output	Pydantic structured output separating observed facts from inferences; multi-provider routing (Groq / Ollama / Mimic).	**100% Satisfied**
Responsible AI & HITL	Deterministic safety policy automatically blocks mutating actions; real LangGraph interrupt() checkpoint pauses for human review.	**100% Satisfied**
Full-Stack Working Demo	FastAPI REST backend connected to React + Vite + TypeScript glassmorphism operator copilot dashboard.	**100% Satisfied**
Testing & Verification	93 passing Pytest test cases covering unit, contract, graph integration, safety policy, API, and E2E scenarios.	**100% Satisfied**

Key End-to-End Demonstration Scenarios

Scenario	Operator Query & Context	Agent Behavior & Decision
Demo 1: Normal (Low Risk)	*'Item X123 is missing from A15-B04. System says 3 units.'* Task: TASK-1001	Retrieves bin A15-B04 + neighbours [A15-B03, A15-B05]. Recommends CHECK_NEIGHBOURING_LOCATION. Completes automatically without interrupt.
Demo 2: Edge Case (Medium Risk)	*'Item X124 is missing at A12-B03 and barcode won't scan.'* Task: TASK-1002	Classifies dual exception (MISSING_ITEM + BARCODE_FAILURE). Retrieves both SOPs. Recommends manual barcode entry before sweep.
Demo 3: High Risk (HITL Interrupt)	*'System says 10 units of X125 but counted 6. Update inventory to 6.'* Task: TASK-1003	Safety policy blocks ADJUST_QUANTITY. Graph pauses at interrupt() checkpoint. Presents modal to supervisor. Resumes cleanly upon review.

Conclusion & Submission Status: PickGuard AI fully satisfies all requirements of the Capstone Project. The repository contains a complete working full-stack implementation, 93 passing unit & integration tests, full documentation, and a modern web application interface.